

네트워크 게임 프로그래밍 Project Progress Report



2023180005 김다은

2023180009 김동규

2023192003 김연수

목차

1. 애플리케이션 기획

- ... 1.1 프로젝트 목표
- ... 1.2 기존 애플리케이션 설명
- ... 1.3 기획 중인 애플리케이션
- ... 1.4 추가 구현 및 필요 기능

2. 하이 레벨 디자인

- ... 2.1 시스템 아키텍처
- ... 2.2 하이 레벨 디자인

3. 로우 레벨 디자인

- ... 3.1 메시지 포맷 정의
- ... 3.2 패킷 프로토콜 설계
- ... 3.3 사용 함수 설계
- ... 3.4 데이터 구조
- ... 3.5 클라이언트 스레드 동기화 방안
- ... 3.6 서버 스레드 동기화 방안
- ... 3.7 네트워크 통신 구조 상세
- ... 3.8 소켓 옵션

4. 역할 분담 및 개발 환경

- ... 4.1 역할 분담
- ... 4.2 개발 환경

5. 개발 일정

1. 애플리케이션 기획

1.1 프로젝트 목표

본 프로젝트의 목표는 기존에 OpenGL 기반으로 개발된 싱글 플레이어 동물 농장 게임 "동물농장"에 네트워크 기능을 추가하여, 여러 사용자가 함께 즐길 수 있는 멀티플레이어 게임으로 확장하는 것입니다. 이를 통해 사용자 간의 상호작용과 농장 운영 속 협력의 재미를 더하고자 합니다.

1.2 기존 애플리케이션 설명

: 동물농장



a. 게임 개요 및 개발자

- 제작자 : 컴퓨터 그래픽스 과목, 2023192003 김연수, 동물농장
- ‘양육간식 동물가족’ 게임에서 영감을 받아 개발한 방치형 힐링 동물 육성 시뮬레이션 게임.
- 플레이어는 자신만의 농장을 운영하며 다양한 동물들을 키우고 팔 수 있으며, 간식을 주며 농장을 관리할 수 있습니다.

b. 주요 기능 및 특징

- 동물 육성 및 관리 : 돼지, 알파카, 병아리, 여우, 펭귄 총 5종의 동물을 지원, 플레이어는 이 동물들을 구매하고 사료를 공급하여 성장시킬 수 있습니다.
- 구매 및 판매 : 상점에서 코인을 사용하여 새로운 동물을 구매할 수 있습니다. 완전히 성장한 동물은 판매하여 코인을 획득할 수 있습니다.
- 환경 변화 : 낮과 밤의 시간 변화가 구현되어 있으며, 주기적으로 눈이 오는 날씨 효과가 적용됩니다.
- 상호작용 : 동물들은 일정 시간 간격으로 배설물을 생성합니다. 플레이어는 이를 수거하여 사료 또는 코인을 획득하고 농장 운영에 활용할 수 있습니다.

c. 기존 개발 구조

현재 게임은 싱글 플레이어 환경을 기반으로 다음과 같은 클래스 구조로 개발되었습니다.

- **Scene** : 게임 화면 렌더링 및 해당 씬의 주요 게임 로직(환경 요소, UI 등) 처리를 담당합니다.
- **GameObject** : 게임 내 객체의 부모입니다. 위치, 회전 등 공통 속성을 관리하며, 객체 업데이트 및 그리기 등의 핵심 기능을 가상 함수 인터페이스로 제공하여 자식 클래스에서 재정의할 수 있도록 합니다.
- **PlayerObject** : GameObject를 상속받아 플레이어 캐릭터를 구현합니다. 키보드/마우스 입력 처리, 캐릭터 이동, 시점 변환 및 월드 내 상호작용 로직을 담당합니다. 현재 플레이어 캐릭터의 시각적 모델링은 구현되지 않았습니다.
- **Animal Class** : GameObject를 상속받아 각 동물의 고유한 특성을 구현합니다. 여기에는 개별 모델링 정보, 이동 AI 로직, 성장 상태(아기/어른, 먹이 횟수) 등이 포함됩니다. Scene을 통해 최종적으로 화면에 렌더링됩니다.
- **SceneManager** : 게임 내 여러 Scene 객체의 생성, 관리 및 전환을 담당합니다.
- **ResourceManager** : 셰이더, 텍스처, 3D 모델(VAO/VBO) 등 게임 전반에서 사용되는 리소스를 로드하고 관리하는 역할을 수행합니다.

1.3 기획 중인 애플리케이션

기존 "동물농장"의 힐링과 육성 경험을 하나의 공유된 농장 공간으로 확장하여, 여러 플레이어가 동시에 접속하여 함께 농장을 가꾸고 상호작용하는 멀티플레이 게임을 기획합니다.

- **실시간 공유 농장** : 모든 플레이어는 동일한 농장 맵에 접속하여 실시간으로 다른 플레이어의 캐릭터와 동물들의 움직임을 확인합니다.
- **공유 동물 상호작용** : 농장 내의 모든 동물에게 누구나 사료를 주거나 배설물을 치울 수 있습니다.
다 자란 동물은 어떤 플레이어든 판매하여 코인을 얻을 수 있습니다. (구매자와 판매자가 다를 수 있음)
- **자원 관리** : 플레이어는 동물 판매, 배설물 수거 등을 통해 얻은 코인과 사료를 개인적으로 소유하고 관리하며, 이를 사용하여 새로운 동물을 구매(농장에 추가)할 수 있습니다.
- **소통 및 식별** : 기본적인 채팅을 통해 다른 플레이어와 대화할 수 있으며, 각 플레이어 캐릭터 머리 위에 표시되는 닉네임으로 서로를 구분합니다.

1.4 추가 구현 및 필요 기능

클라이언트-서버 간 실시간 데이터 교환을 위해 TCP/IP 기반의 소켓 통신 프로그래밍을 구현합니다.

a. 클라이언트-서버 구조 도입

- **서버** : 하나의 공유 농장 상태(동물 목록, 위치, 성장 상태 등)와 개별 플레이어 상태(위치, 코인, 사료 등)를 중앙에서 관리하고 게임 로직을 처리하는 서버 프로그램을 개발합니다.
- **클라이언트** : 사용자 입력을 서버로 전송하고, 서버로부터 공유 농장 및 모든 플레이어의 최신 상태를 받아 시각적으로 렌더링하는 역할에 집중하도록 기존 코드를 수정합니다.

b. 네트워크 통신 및 서버 동기화 기능

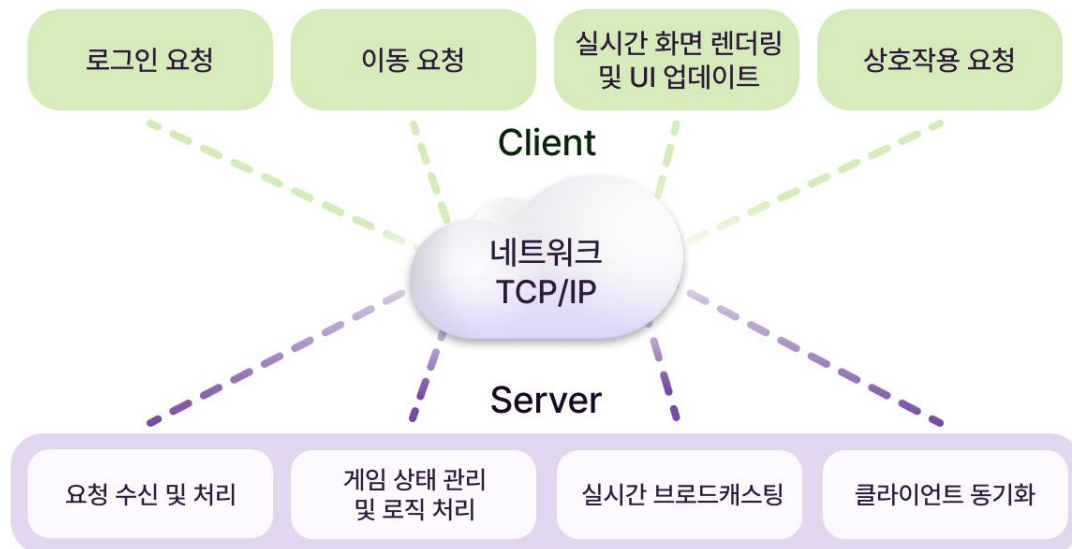
- 공유 상태 관리 : 기존 gameScene에 있던 동물 관련 모든 데이터(목록, 위치, 성장 상태 등) 및 농장 환경 상태(낮/밤, 날씨)를 서버에서 통합 관리합니다.
- 개별 상태 관리 : 코인, 사료 등 플레이어 개인 자원은 서버에서 플레이어별로 구분해 관리합니다.
- 입력 처리 및 동기화 : 클라이언트의 모든 행동은 서버로 요청 형태로 전송, 서버는 클라이언트 요청의 유효성을 검사하고, 상태를 변경한 뒤 결과를 모든 클라이언트에게 실시간 브로드 캐스팅을 통해 알립니다.
- 서버 시뮬레이션 : 동물 이동 AI, 성장, 배설물 생성, 시간/날씨 변화 로직을 서버에서 주기적으로 실행합니다.
- 다중 플레이어 렌더링 : 클라이언트는 서버로부터 받은 데이터를 기반으로 자신을 포함한 모든 플레이어 캐릭터와 모든 동물을 화면에 그려야 합니다.

c. 추가 기능 구현

- 채팅 시스템 : 서버를 경유해 모든 클라이언트에게 메시지를 전달하는 기본적인 텍스트 채팅
- 닉네임 표시 : 서버로부터 플레이어 ID와 닉네임 정보를 받아, 각 플레이어 모델의 머리 위에 3D 텍스트 또는 빌보드 형태로 닉네임을 렌더링하는 기능을 구현합니다.

2. 하이 레벨 디자인 (High Level Design)

2.1 시스템 아키텍처



본 게임 동물농장은 실시간 멀티플레이 환경을 지원하기 위해 클라이언트-서버(Client-Server) 아키텍처 모델을 기반으로 설계됩니다. 서버는 게임 데이터 관리 및 동기화를 담당하고, 클라이언트는 사용자 입력과 화면 렌더링을 담당합니다.

a. 게임 서버

- 모든 클라이언트의 로그인 연결 관리
- 공유 농장 상태(동물 목록, 위치, 성장 상태 등) 및 개별 플레이어 상태(위치, 코인, 사료 등)의 데이터를 관리.
- 게임 로직 시뮬레이션 (동물 이동 로직, 시간/날씨 변화, 배설물 생성 등) 및 채팅-중개
- 클라이언트로부터 수신된 요청(이동, 상호작용) 처리 및 유효성 검증.
- 모든 상태 변경 사항을 관련된 클라이언트들에게 전파(Broad Casting).

b. 게임 클라이언트

- 게임 서버에 접속 및 연결 유지
- 사용자의 키보드/마우스 입력을 감지하여 서버로 해당 요청 메시지를 전송.
- 서버로부터 수신된 게임 상태 업데이트(자신, 다른 플레이어, 동물, 환경 등)를 실시간으로 반영하여 3D 월드를 렌더링.
- 채팅 메시지 수신, 플레이어 닉네임 등 UI 표시.

2-2. 하이 레벨 디자인

(a). 로그인 및 접속 처리

1. **클라이언트** : 게임 실행 후, 서버 주소로 접속을 시도하고, 성공 시 닉네임 입력 후 서버로 로그인 요청 메시지를 전송
2. **서버** : 로그인 요청 메시지 수신 시, 해당 닉네임이 유효한지 확인 후 로그인 수락
3. **서버** : 새로운 플레이어의 접속 정보를 다른 모든 플레이어에게 브로드 캐스팅
4. **서버** : 접속한 클라이언트에게는 ‘로그인 성공’ 이라는 응답과 함께 현재 농장의 전체 상태 및 해당 플레이어의 개인 정보 전송
5. **클라이언트** : ‘로그인 성공’ 및 초기 데이터 수신 후 수신된 데이터 기반으로 월드 및 다른 플레이어들 렌더링

(b). 게임 진행 - 플레이어 이동 동기화

1. **클라이언트** : 사용자가 이동 키를 누르면, 이동 방향을 계산해 서버로 ‘이동 요청’ 메시지 전송
2. **서버** : 특정 플레이어로부터 ‘이동 요청’ 수신
3. **서버** : 요청된 방향으로 이동 시 새로운 위치를 계산, 충돌여부를 검사해 해당 플레이어의 위치 상태를 갱신
4. **서버** : 변경된 플레이어의 최신 위치 및 방향 정보를 모든 클라이언트에게 브로드 캐스팅
5. **클라이언트** : 브로드 캐스팅을 통해 받은 새로운 데이터를 통해 플레이어의 화면의 위치와 방향 갱신

(c). 게임 진행 - 동물 상호작용

1. **클라이언트** : 플레이어가 움직일 때 마다 동물 근처라면, 자신이 근처라는 메시지를 전송
2. **서버** : 받은 메시지를 통해 진짜 근처인지 계산 후, 근처라면 상호작용이 가능함을 전송
3. **클라이언트** : 상호작용이 가능하다는 메시지가 오면, UI를 띄운 후 다시 상호작용을 위한 키 입력 후 ‘상호작용 요청’ 메시지 전송
4. **서버** : 요청 수신 후, 유효성을 검사한 뒤 해당 동물의 상태를 업데이트 후 플레이어의 사료 개수를 감소시킴.
5. **서버** : 변경된 동물 상태와 플레이어의 자원 상태를 모든 클라이언트에게 상태 업데이트 메시지로 브로드 캐스팅
6. **클라이언트** : 받은 데이터를 통해 플레이어의 자원 상태 UI 업데이트

~~(d). 채팅 기능~~

- ~~1. 클라이언트 : 플레이어가 채팅 메시지를 입력하고 전송~~
- ~~2. 클라이언트 : 채팅 메시지 전송 요청을 서버에 전송~~
- ~~3. 서버 : 메시지를 수신 후 모든 클라이언트에게 채팅 메시지를 브로드 캐스팅~~
- ~~4. 클라이언트 : 메시지 수신 후, 채팅 UI에 메시지를 표시~~

(e). 게임 진행 2 - 환경 변화 및 동기화

1. 서버 (GameLoop) : 서버의 환경 관리자(EnvironmentManager)는 메인 게임 루프에서 경과 시간(DeltaTime)을 누적하여 낮/밤 및 날씨(눈)타이머를 지속적으로 업데이트합니다.
2. 서버 : 각 타이머가 설정된 주기에 도달하면 환경 상태 변수(isDay, isSnow)를 갱신하고, 즉시 변경된 상태 정보를 담은 패킷(SC_ENVIRONMENT_UPDATE)을 생성합니다.
3. 서버 : 생성된 환경 패킷을 접속 중인 모든 클라이언트에게 실시간으로 브로드캐스팅(Broadcast)합니다.
4. 클라이언트 : 서버로부터 환경 업데이트 패킷을 수신하면, 즉시 렌더링 상태를 변경하여 시각적 효과(낮/밤 전환, 눈 파티클 활성화/비활성화)를 게임 화면에 반영합니다.
5. 서버 (접속 시) : 새로운 클라이언트가 로그인 요청을 하면, 서버는 현재 진행 중인 환경 상태(낮/밤, 날씨)를 패킷에 담아 전송함으로써 접속과 동시에 기존 유저들과 동일한 환경을 공유하도록 동기화합니다.

3. 로우 레벨 디자인 (Low Level Design)

3-1. 메시지 포맷 정의

```
enum class MessageType {
    CS_LOGIN_REQUEST, SC_LOGIN_SUCCESS, SC_LOGIN_FAILURE,
    SC_PLAYER_JOIN, SC_PLAYER_LEAVE,
    CS_MOVE_REQUEST, SC_STATE_UPDATE,
    CS_INTERACT_REQUEST, SC_INTERACT_RESULT,
    CS_CHAT_MESSAGE,
    // ... 기타 메시지 타입
};

struct MessageHeader {
    MessageType type;
    unsigned char size;
};

struct packet {
    MessageHeader h;
    // 각 패킷에 맞는 필요 데이터
};
```

클라이언트와 서버가 주고 받을 데이터의 구조인 모든 메시지는 패킷 ID(정수형) 와 데이터 사이즈를 포함하는 내용으로 시작합니다. 수신 측에서는 이 정보를 통해 메시지의 종류를 식별하고, 이어지는 실제 데이터를 정확한 크기만큼 읽어들이니다.

3-2. 패킷 프로토콜 설계

- SC_ = 서버(S) -> 클라이언트(C)
- CS_ = 클라이언트 (C) -> 서버 (S)

패킷 ID	이름	방향	전달 데이터	설명
1000	CS_LOGIN_REQUEST	C → S	UserID	해당 ID 로 접속요청
2000	SC_LOGIN_ACCEPT	S → C		로그인 성공
2001	SC_LOGIN_FAIL	S → C		로그인 실패
2002	SC_LOGIN_NOTIFY	S → C	플레이어 데이터	클라이언트 초기 설정
2007	SC_LOGOUT	S → C		로그아웃

2004	SC_SPAWN_ANIMAL	S → C	AnimalID, GrowStep, X, Y	동물 생성
2005	SC_SPAWN_POOP	S → C	PoopID, X, Y	똥 생성
1001	CS_MOVE	C → S	X, Y	이동 요청
2003	SC_UPDATE_MOVE	S → C	PlayerID, X, Y	다른 플레이어의 이동 업데이트(위치 동기화)
2006	SC_STAT_CHANGE	S → C		플레이어 자원 업데이트
1050	CS_REQUEST_CLEAN_POOP	C → S	PoopID	똥 청소 요청
2051	SC_REMOVE_POOP	S → C	PoopID	해당 똥 제거
1060	CS_REQUEST_BUY_ANIMAL	C → S	AnimalType	동물 구매 요청
1070	CS_REQUEST_FEED	C → S	AnimalID	동물 사료 급여 요청
2008	SC_UPDATE_ANIMAL_STATE	S → C	AnimalID, GrowStep	모든 클라이언트에게 동물 성장 단계 동기화
2009	SC_ANIMAL_COUNT	S → C	AnimalCount	동물 마릿수 전달
2010	SC_REMOVE_ANIMAL	S → C	AnimalID	동물 삭제 요청
2011	SC_MOVE_ANIMAL	S → C	AnimalID, X, Y	동물 위치 이동
1080	CS_REQUEST_SELL	C → S	AnimalID, AnimalType	동물 판매 요청
2052	SC_ENVIRONMENT_UPDATE	S → C	isDay, isSnow	날씨, 시간 변수 전달
1090	CS_ENTER_FARM	C → S		플레이어 농장 입장
1091	CS_OUT_FARM	C → S		플레이어 농장 퇴장

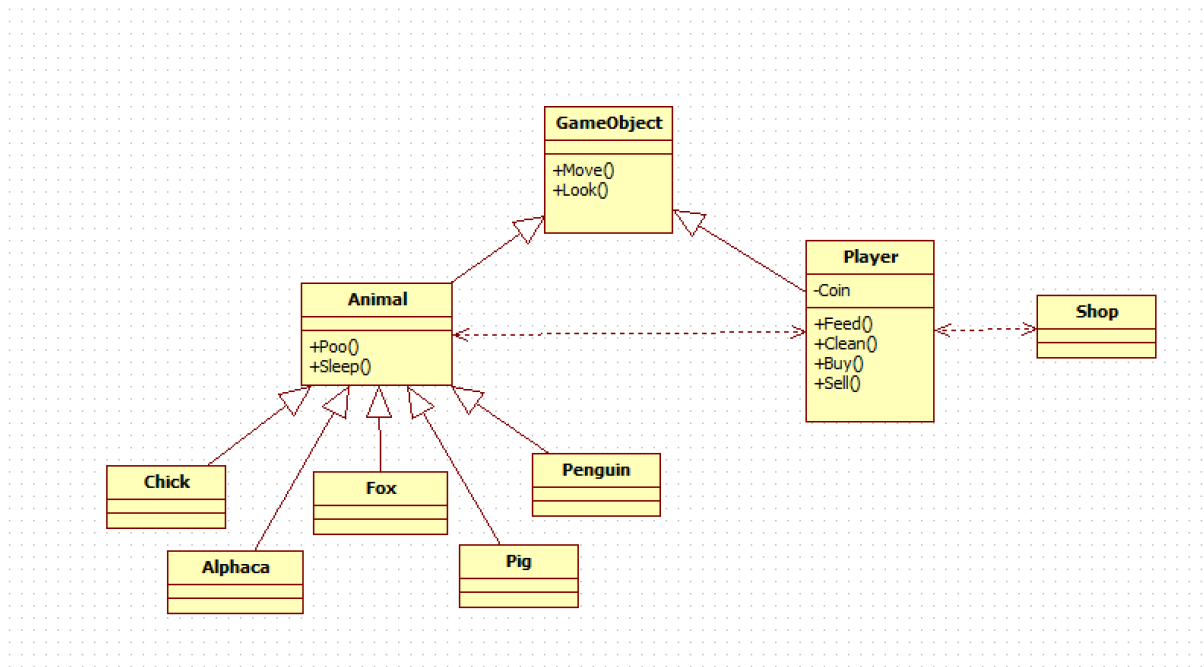
3-3. 함수 설계

함수 이름	사용할 패킷	함수 설명
Login()	CS_LOGIN_REQUEST	플레이어 - 해당 ID 로 접속요청
LoginFailed()	SC_LOGIN_FAILED	서버 - 로그인 실패 메서지전송
LoginAccepted()	SC_LOGIN_RESPONSE	서버 - 접속 허용 및 클라이언트 자원 설정
SpawnPlayer()	SC_SPAWN_PLAYER	서버 - 플레이어 스폰 설정
SpawnAnimal()	SC_SPAWN_ANIMAL	서버, 클라 - 동물 생성
SpawnPoop()	SC_SPAWN_POOP	서버, 클라 - 똥 생성
Move()	CS_MOVE	플레이어 - 이동 요청
UpdateMove()	SC_UPDATE_MOVE	서버 - 다른 플레이어의 이동 업데이트(위치 동기화)
CompleteCleaning()	SC_RESULT_CLEAN_POOP	서버 - 요청한 클라이언트에게 사료를 지급
RemovePoop()		서버, 클라 - 해당 똥 제거
BuyAnimal()	CS_REQUEST_BUY_ANIMAL	플레이어 - 동물 구매 요청
CompletePurchase()	SC_RESULT_BUY_ANIMAL	서버 - 구매 요청한 클라이언트의 코인에서 동물 구매 비용 차감
FeedAnimals()	CS_REQUEST_FEED_ANIMAL	플레이어 - 동물 사료 급여 요청
UpdateAnimalState()	SC_UPDATE_ANIMAL_STATE	서버 - 모든 클라이언트에게 동물 성장 단계 동기화
SellAnimal()	CS_REQUEST_SELL_ANIMAL	플레이어 - 동물 판매 요청

CompleteSetting()	SC_RESULT_SELL_ANIMAL	서버 — 요청한 클라이언트의 코인에 판매아이템 추가
RemoveAnimal()	SC_REMOVE_ANIMAL	(모든 클라이언트에게) 서버 - 동물을 삭제
sendPacket()		클라 - 서버에게 패킷 보내는 함수
ProcessPacket()	-	서버&클라이언트 - 패킷 처리
InitClient()		클라이언트 - 클라이언트 초기화
CreateThread()		스레드 생성 함수
ConnectServer()		클라이언트 — 서버 연결
InitServer()		서버 - 서버 초기화
AcceptClient()		서버 — 클라이언트 접속 허용
DisconnectClient()		서버 — 클라이언트 접속 종료
ShowUI()	-	클라이언트 — UI 구현
ReceiveThread()		클라이언트 - 서버와의 네트워크 처리 스레드
ClientThread()		서버 - 클라이언트 스레드 생성 및 네트워크 처리
GameLoopThread()		서버 - 서버 내 게임루프 전담 스레드
GameLoop()		서버 - 게임루프 스레드에서 작동하는 함수
UpdateGameWorld()		서버 - 게임 로직 계산
NetworkThreadFunc()		네트워크 연결 스레드 생성
InitializeCriticalSection()		임계 영역 객체를 사용하기 전 초기화
DeleteCriticalSection()		사용이 완료된 임계 영역 객체를 삭제하고 리소스 해제
EnterCriticalSection()		공유 자원 접근 전 호출 (다른 스레드가 사용 중이면 대기)
LeaveCriticalSection()		공유 자원 접근 완료 후 호출 (잠금 해제)
WSACreateEvent()		WSAEventSelect와 함께 사용할 이벤트(Event) 객체를 생성
SetEvent()		서버 - 이벤트를 신호상태로 변환
BroadcastAnimalState()		서버 - 동물 상태 브로드캐스팅

animalUpdate()		서버 – 동물 이동 업데이트
broadcastMovement()		서버 – 동물 이동 브로드 캐스팅
SendExistingObjects()		서버-접속한 유저에게 현재 나와있는 동물/똥 목록 전송
SendCurrentEnvironment()		서버 – 현재 환경(시간, 날씨) 정보 전송
GetNextAnimalID()		서버 – 다음 동물 id 가져오는 함수
IsSnow()		서버 – 현재 날씨 상태 get 함수
IsDay()		서버 – 현재 시간 상태 get 함수
BroadcastEnvironment()		서버 – 현재 환경 브로드 캐스팅 함수
GetNextPoopID()		서버 – 다음 똥 id 가져오는 함수
gorwAnimal()		서버 – 동물 성장 업데이트 함수
checkInteraction()		클라이언트 – 동물과 똥 상호작용 체크
updateAnimalPos()		클라이언트 – 동물 이동 업데이트 함수
EnqueueAnimalMove()		클라 – 수신 스레드에서 큐 넣는 함수
ProcessAnimalMoveQueue ()		클라 – 메인 스레드에서 큐에 든 내용 처리 함수
setEnvironment()		클라 – 환경 업데이트 함수
DrawTextWithAtlas()		클라 – 텍스처 uv 조정해 한글자씩 렌더링하는 함수
renderName()		클라 – 플레이어 이름 렌더링 함수
createOtherPlayer		클라 – 접속한 다른 플레이어 생성
removeOtherPlayer		클라 – 로그아웃한 다른 플레이어 삭제
updateOtherPlayer		클라 – 이동하는 플레이어 업데이트 함수
setAnimalCount()		클라 – 동물 마릿 수 업데이트 함수
WSAWaitForMultipleEvent s()		지정된 이벤트 객체 중 하나라도 신호상태가 될 때까지 스레드 대기

3-4. 데이터 구조



-플레이어 관리 (PlayerManager)

- 접속 중인 각 플레이어의 상태를 관리하기 위해 `std::map<int, PlayerData>` (또는 `std::vector<PlayerData>`) 형태의 컨테이너를 사용합니다. 이때 `int`는 각 플레이어에게 부여된 고유 `id`

- 플레이어 데이터 관리

- 관리 방식: 접속 중인 클라이언트의 세션 정보를 관리하기 위해 `std::map<SOCKET, PlayerSession>` 형태의 전역 컨테이너(`g_sessions_map`)를 사용합니다. 이때 `Key값인 SOCKET`은 클라이언트의 고유 핸들.

• PlayerData 구조체

• PlayerSession 구조체

- sock : 해당 클라이언트와의 통신 소켓 핸들
- playerId : 고유 식별자
- clientSocket : 해당 클라이언트와의 통신 소켓 정보
- nickname : 플레이어 닉네임 (ID로 통일)
- x, y, coinNum, feedNum : 플레이어 기본 위치와 주요 자원

~~- 동물 관리 (AnimalManager)~~

- ~~• 농장에 존재하는 모든 동물의 상태를 관리하기 위해 `std::vector<AnimalData>` (또는 `std::map<int, AnimalData>`) 형태의 컨테이너를 사용합니다. 각 동물은 고유 ID를 가집니다.~~

~~• AnimalData 구조체~~

- ~~—— animalId : 고유 식별자~~
- ~~—— animalType : 동물의 종류를 나타내는 타입~~
- ~~—— x, y, dir : 동물의 현재 좌표 및 방향~~
- ~~—— feedNum, isBaby : 동물의 성장상태를 나타내는 정보들~~

- 동물 및 오브젝트 관리 (AnimalManager)

- 농장에 존재하는 모든 동물의 상태를 관리하기 위해 `std::vector<AnimalData>` 형태의 컨테이너를 사용합니다. 각 동물은 고유 ID를 가집니다.

• AnimalData 구조체

- id: 동물 고유 식별자
- type: 동물의 종류 (돼지, 펭귄 등 Enum 타입)
- x, y: 동물의 현재 위치
- xDir, yDir: 동물의 이동 방향 벡터
- growStep: 동물의 성장 단계를 나타내는 정수값 (0 부터 시작)

- PoopData 구조체 (추가됨)

- id: 배설물 고유 식별자
- x, y: 배설물이 생성된 위치

3-5. 클라이언트 스레드 동기화 방안

적용할 스레드 동기화 기법 - 임계영역, 스레드 분리 및 메시지 큐

(a) Critical Section (임계 영역)

~~—Critical Section은 PlayerManager, AnimalManager와 같이 여러 스레드가 동시에 접근할 수 있는 공유 데이터(Shared Data)를 보호하기 위해 사용됩니다. 이는 만약 1번 스레드가 '동물 구매' 패킷을 처리하며 AnimalManager에 동물을 추가하는 도중에, 2번 스레드가 '동물판매' 패킷을 받아 AnimalManager에서 동물을 삭제하려 하는 경쟁 상태를 방지합니다.~~

~~이를 구현하기 위해, AnimalManager와 같은 공유 자원에 접근하는 코드의 시작 지점에서 EnterCriticalSection(&cs)을 호출하고, 접근이 종료되는 지점에서 LeaveCriticalSection(&cs)을 호출하여 데이터 접근을 직렬화하고 데이터의 무결성을 보장합니다.~~

- 수신 스레드와 메인 스레드가 데이터를 주고받는 통로인 패킷 큐의 데이터 무결성을 보호하기 위해 사용합니다.
- std::mutex와 std::lock_guard를 사용하여, 큐에 데이터를 넣거나(push) 꺼낼 때(pop) 발생할 수 있는 경쟁 상태를 방지했습니다.

(b) Event 동기화

~~—이벤트 동기화 기법은 클라이언트의 게임 로직/렌더링과 네트워크 통신 작업을 분리하여, 네트워크 작업이 게임 화면을 멈추게 하는 현상을 방지하기 위해 사용됩니다.~~

~~이를 위해 메인 스레드(게임 루프, 렌더링 담당)와 네트워크 스레드(데이터 수신 담당)을 분리한다.~~

(b) 스레드 분리 및 메시지 큐

- 네트워크 통신이 게임 화면(렌더링)을 멈추게 하는 현상을 방지하고, 멀티스레드 환경에서의 메모리 충돌을 차단하기 위해 메시지 큐 구조를 도입했습니다.
- 메인 스레드(게임 루프, 렌더링 담당)와 네트워크 스레드(데이터 수신 담당)을 분리했습니다.

1. 네트워크 스레드 (수신부)

- ~~→ WSAEventSelect 함수를 사용하여 클라이언트 소켓에 FD_READ(데이터 수신) 및 FD_CLOSE(연결 종료)와 같은 네트워크 이벤트를 등록하고, 이를 WSAEVENT 객체와 연결합니다.~~
- ~~→ 네트워크 스레드는 루프 내에서 WSAWaitForMultipleEvents 함수를 호출하여, 이 이벤트 객체가 신호(Signaled) 상태가 되기를 효율적으로 대기합니다.~~
- ~~→ 서버로부터 데이터가 도착하면 운영체제(OS)가 해당 이벤트를 '신호 상태'로 변경해줍니다.~~

어때 대기 중이던 네트워크 스레드가 즉시 활성화되어 `recv()`를 호출하고 데이터를 안전하게 수신합니다.

- 메인 스레드와 분리된 별도의 워커 스레드(ReceiveThread)를 생성하여 서버와의 통신을 전담합니다.
- 블로킹(Blocking) 소켓 방식을 사용하여 `recv()` 함수를 호출하고, 서버로부터 데이터가 도착할 때까지 대기합니다.

- 데이터(패킷)가 수신되면, 게임 로직을 직접 처리하지 않고 즉시 Critical Section 으로 보호된 공유 패킷 큐에 데이터를 적재(Enqueue)하고 다시 대기 상태로 돌아갑니다.

2. 메인 스레드 (게임 루프)

— 네트워크 스레드가 이벤트 신호를 기다리며 대기하는 동안에도, 메인 스레드는 멈추지 않고 지속적으로 게임 로직을 처리하고 화면을 렌더링합니다.

- 게임 로직 연산(클라이언트 측 상태 반영)과 화면 렌더링을 담당하는 메인 스레드는 네트워크 상황과 관계없이 독립적인 프레임을 유지합니다.
- 메인 스레드는 매 프레임의 `Update()`의 시작 부분에서 이 큐를 확인하여, 수신된 패킷이 있다면 이를 꺼내어 게임 월드(Game World)에 반영합니다.

3-6. 서버 스레드 동기화 방안

1. 클라이언트 당 스레드 모델 (Thread-per-Client)

(a) 메인 스레드 (Listener)

- 서버의 메인 스레드는 listen() 이후 accept() 함수를 호출하여, 새로운 클라이언트의 접속을 대기하는 '리스너(Listener)' 역할만 전담합니다.

(b) 워커 스레드 (클라이언트 전담 스레드)

- accept()를 통해 클라이언트의 접속이 성공하면, 메인 스레드는 즉시 해당 클라이언트와의 1:1 통신을 전담할 워커 스레드를 생성합니다.
- 메인 스레드는 스레드 생성 직후, 다시 accept() 대기 상태로 돌아가 다음 접속을 준비합니다.

(c) 게임 루프 스레드 (Game Loop Thread)

- 통신과 별개로 독립된 스레드(GameLoopThread)가 하나 존재하여, 1 초에 SERVER_TICK_RATE 씩 게임 월드를 주기적으로 업데이트합니다.

2. 통신 방식: 블로킹(Blocking) 소켓 및 동기화 기법

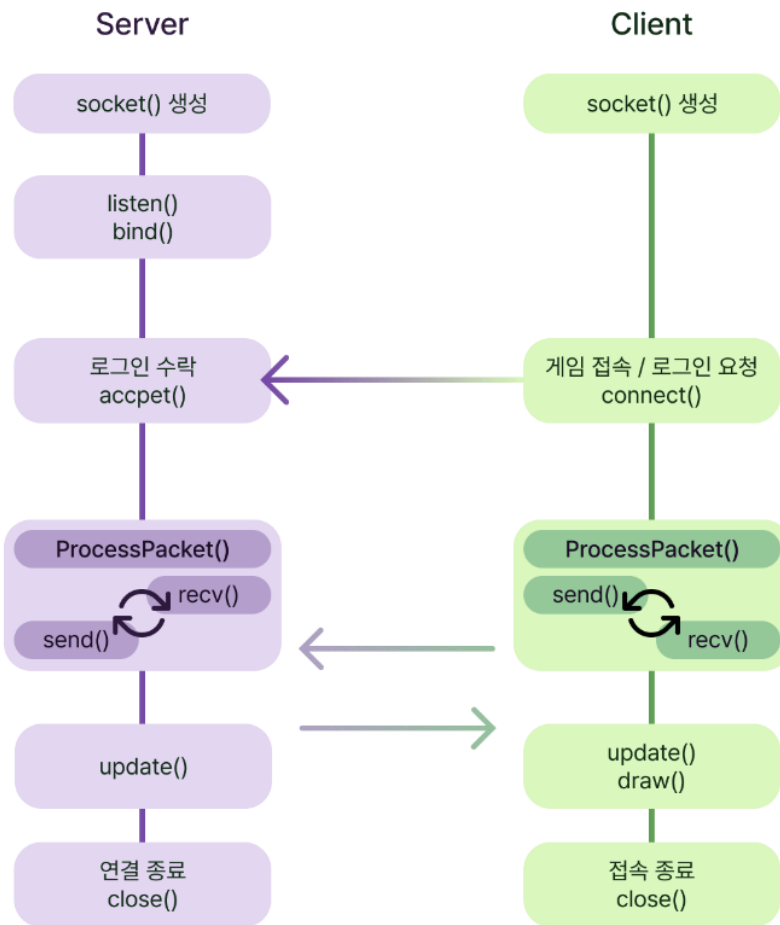
(a) 블로킹 recv() 사용

- 각 워커 스레드는 블로킹(Blocking) 방식의 recv() 함수를 호출하여, 자신이 전담한 클라이언트로부터 데이터가 올 때까지 대기합니다.
- 한 스레드(A 유저 담당)가 recv()에서 '블로킹(멈춤)' 상태가 되어도, 다른 스레드(B 유저, C 유저 담당)는 각자 독립적으로 실행되므로 서버 전체 성능에 영향을 주지 않습니다.

(b) 임계 영역 (Critical Section)

- AnimalManager, PlayerManager 및 게임 월드맵 정보 등, 모든 워커 스레드가 공동으로 접근해야 하는 서버의 공용 데이터가 대상
- 공용 데이터에 접근하는 모든 코드의 시작과 끝을 EnterCriticalSection과 LeaveCriticalSection으로 보호하여 데이터의 무결성을 보장

3-7. 네트워크 통신 구조 상세



서버

1. 초기화 : WSASStartup()
2. 소켓 생성 : socket()
3. 주소 할당 및 연결 대기 : bind(), listen()
4. 클라이언트 연결 수락 (반복문) : accept()
5. 데이터 수신 및 송신 : recv(), send(), broadcast()
6. 연결 종료 : closesocket(), WSACleanup()

클라이언트

1. 초기화 : WSASStartup()
2. 소켓 생성 : socket()
3. 서버 연결 : connect()
4. 데이터 수신 및 송신 : recv(), send()
5. 화면 그리기 : draw()
6. 연결 종료 : closesocket(), WSACleanup()

3-8. 소켓 옵션

안정적인 서버 운영과 효율적인 개발 환경 구축을 위해, 서버의 Listen Socket에 다음과 같은 소켓 옵션을 적용하였습니다.

(1) SO_KEEPALIVE

- 적용 목적 : 클라이언트의 비정상적인 종료(강제 종료, 네트워크 단절 등)로 인해 발생하는 불필요한 시스템 자원 소모를 막기 위함.
- 구현 내용 : 대기 소켓이 적용하여 대기 소켓이 TCP 프로토콜 차원에서 주기적으로 연결 상태를 확인하는 TCP패킷을 보내도록 설정하였습니다. 클라이언트로부터 응답이 없으면 연결을 자동으로 종료시켜 불필요한 시스템 자원 소모를 막습니다.

(2) SO_REUSEADDR

- 적용 목적 : 현재 사용 중인 IP 주소와 포트 번호를 재사용할 수 있기 하기 위함.
- 구현 내용 : 서버가 종료된 후에도 TCP 규약에 따라 포트가 일정 시간 동안 TIME_WAIT 상태로 남아있게 되는데, 이 옵션을 활성화하여 해당 포트가 TIME_WAIT 상태일지라도 즉시 재사용하여 서버를 바로 다시 시작할 수 있도록 하였습니다.

4. 역할 분담 및 개발 환경

4.1 역할 분담

a. 김다은

- 클라이언트 통신 로직 구현
- 서버 connect 및 send/recv 루프 구현
- 실시간 객체 상태 동기화 로직 구현 (브로드 캐스팅)
- ~~- 클라이언트로부터 받은 패킷 분류 및 처리 함수 구현~~
- 로그인, 채팅, 상점, 상호작용 등 개별 기능의 서버 로직 및 네트워크 처리 구현

b. 김동규

- 서버 프로그램 기본 구조 설계
- TCP 소켓 (listen, accept) 통한 클라이언트 연결 관리
- 클라이언트로부터 받은 패킷 분류 및 처리 함수 구현

c. 김연수

- 기존 싱글 플레이 게임 코드 리팩토링
- ~~- 로그인, 채팅, 상점, 상호작용 등 개별 기능의 서버 로직 및 네트워크 처리 구현~~
- 클라이언트 UI/UX 구현 및 서버 요청 발생 로직 구현
- 서버에서 전달된 데이터 기반으로 화면 렌더링
- 클라이언트로부터 받은 패킷 분류 및 처리 함수 구현

4.2 개발 환경

- 프로그래밍 언어 : C++
- 개발 환경 :
 - OS : Windows 10/11
 - IDE : Visual Studio 2022
 - 그래픽스 API : OpenGL
- 버전 관리 시스템 : Github

5. 개발 일정

5.1 개발 일정 – 김다운

Mon	Tue	Wed	Thu	Fri	Sat	Sun
10/27	10/28	10/29	10/30	10/31	11/1	11/2
				클라이언트 코드 정리	InitClient() 및 프로젝트 분리	
11/3	11/4	11/5	11/6	11/7	11/8	11/9
소켓 생성, ConnectServer() 구현		Login 패킷 구현, Login() 구현			수신된 이동 정보 파싱 및 브로드캐스팅	[팀] 1 차 통합 (로그인 기능 테스트)
11/10	11/11	11/12	11/13	11/14	11/15	11/16
Event 동기화 (클라이언트의 게임 로직과 네트워크 통신 작업 분리)	FeedAnimal() 요청 처리 / 브로드캐스팅 마름		UpdateMove() 요청 후 다른 플레이어 위치/방향 갱신 로직 구현	2 차 통합 (다중 접속 렌더링) (이동동기화)		
11/17	11/18	11/19	11/20	11/21	11/22	11/23
		UpdateAnimal Pos() 구현/ 브로드캐스팅		BuyAnimal() 요청 처리/ 브로드캐스팅	[팀] 3 차 통합 (이동동기화) (다중 접속 렌더링)	[팀] 3 차 통합 버그 수정 및 추진 계획서 수정
11/24	11/25	11/26	11/27	11/28	11/29	11/30
	SellAnimal() 처리 요청 구현 및 removeAnimal() 구현	FeedAnimal() 요청 처리 / 브로드캐스팅	CleanPoop() 로직 및 브로드 캐스팅		[팀] 4 차 통합 (동물/환경 동기화) 동물 init 클라 초기 세팅 추가	[팀] 4 차 통합 버그 수정 및 추진 계획서 수정
12/1	12/2	12/3	12/4	12/5		
클라 로직 오류 수정			[팀] 전체 기능 통합 및 버그 수정 [팀] 최종 리허설 및 테스트	[팀] 프로젝트 최종 점검 및 제출		
12/8	12/9	12/10	12/11	12/12		

5.2 개발 일정 – 김동규

Mon	Tue	Wed	Thu	Fri	Sat	Sun
10/27	10/28	10/29	10/30	10/31	11/1	11/2
				InitServer() 및 프로젝트 분리		
11/3	11/4	11/5	11/6	11/7	11/8	11/9
	AcceptServer() 구현		LoginFailed(),L oginAccepted() 구현	DisconnectCli ent() 구현		[팀] 1 차 통합 (로그인 기능 테스트)
11/10	11/11	11/12	11/13	11/14	11/15	11/16
Client 별 스레드 분리		SetEvent(), 이벤트 동기화		2 차 통합 (다중 접속 렌더링) (이동동기화)		
11/17	11/18	11/19	11/20	11/21	11/22	11/23
공유 자원 구현 (AnimalMan ager) 및 스레드동기화	SpawnAnimal() 구현	SpawnPoop() 구현			[팀] 3 차 통합 (이동동기화) (다중 접속 렌더링)	[팀] 3 차 통합 버그 수정 및 추진 계획서 수정
11/24	11/25	11/26	11/27	11/28	11/29	11/30
		CompleteClea ning()구현	RemovePoop() 구현		[팀] 4 차 통합 (동물/환경 동기화) 똥 동기화 구현	[팀] 4 차 통합 버그 수정 및 추진 계획서 수정
12/1	12/2	12/3	12/4	12/5		
Environment alManager 구현	CompletePurch ase(), UpdateAnimalSt ate() 구현	소켓 옵션 추가	CompleteSelli ng()구현	[팀] 프로젝트 최종 점검 및 제출		
	날씨 동기화		[팀] 전체 기능 통합 및 버그 수정 [팀] 최종 리허설 및 테스트			
12/8	12/9	12/10	12/11	12/12		

5.3 개발 일정 – 김연수

Mon	Tue	Wed	Thu	Fri	Sat	Sun
10/27	10/28	10/29	10/30	10/31	11/1	11/2
				싱글플레이 코드 리팩토링	싱글플레이 코드 리팩토링	
11/3	11/4	11/5	11/6	11/7	11/8	11/9
loginScene 구현 및 UI 제작				ProcessPacket() - 로그인 응답 로직	로그인 응답 처리 및 화면 렌더링	[팀] 1 차 통합 (로그인 기능 테스트)
11/10	11/11	11/12	11/13	11/14	11/15	11/16
Event 동기화 (클라이언트의 게임 로직과 네트워크 통신 작업 분리)			플레이어 모델 구현 <i>미룸</i>	2 차 통합 (다중 접속 렌더링) (이동동기화)		게임 루프 스레드 생성
UI 추가						
11/17	11/18	11/19	11/20	11/21	11/22	11/23
PlayerManager() 구현 <i>이미 Client 로 구현됨으로 구현 X</i>		플레이어 모델 구현		Client - 업데이트 된 데이터에 따른 화면 갱신	[팀] 3 차 통합 (이동동기화) (다중 접속 렌더링)	[팀] 3 차 통합 버그 수정 및 추진 계획서 수정
서버로 자원 관리 권한 이동						로그인 로그아웃 브로드 캐스팅 수정/클라 동기화, 각 클라 회전값도 브로드 캐스팅
11/24	11/25	11/26	11/27	11/28	11/29	11/30
Client - 동물 상호작용 로직 및 showUI() 구현			기존 UI 변경	Client - 업데이트 된 데이터에 따른 화면 갱신 로그인 화면 비율 보정 및 객체량 상호작용 우선순위 적용 및 중복 렌더링 수정	[팀] 4 차 통합 (동물/환경 등커화)	[팀] 4 차 통합 버그 수정 및 추진 계획서 수정
12/1	12/2	12/3	12/4	12/5		
상점 상호작용 및 상점 showUI() 구현			플레이어 농장 총돌체크 및 입 퇴장 적용	[팀] 프로젝트 최종 점검 및 제출		
동물 이동 동기화 스레드 총돌 수정, Client - 플레이어 별 닉네임 표시 및 동기화			[팀] 전체 기능 통합 및 버그 수정 [팀] 최종 리허설 및 테스트			
12/8	12/9	12/10	12/11	12/12		